

Configuration Reference

All configuration lives in the `config/` folder as plain JSON files:

File	Purpose	Edit by hand?
<code>config.template.json</code>	Master template with your station defaults. Copied to <code>config.json</code> on first run.	Yes (before first run)
<code>config.json</code>	The live configuration the app actually uses. Auto-created from the template.	Yes (after first run)
<code>themes.json</code>	Saved color palettes and the active theme.	Usually via the UI
<code>users.json</code>	User accounts (passwords are bcrypt-hashed).	Managed via the UI

After editing `config.json`, **restart the server** for hardware/connection changes (serial, UDP, MQTT, FlexRadio) to take effect.

site — Web server & security

```
"site": {  
  "site_name": "WB0Z - Flex 8600",  
  "station_callsign": "WB0Z",  
  "http_port": 3000,  
  "bind_address": "0.0.0.0",  
  "jwt_secret": "CHANGE_ME_TO_A_LONG_RANDOM_STRING",  
  "jwt_expiry": "12h"  
}
```

Key	Meaning
site_name	Title shown in the top bar and login page.
station_callsign	Your callsign, shown in the header.
http_port	TCP port the web UI/API listen on. Change if 3000 is taken.
bind_address	0.0.0.0 = reachable on the whole LAN; 127.0.0.1 = local only.
jwt_secret	Secret used to sign login tokens. Leave the CHANGE_ME... placeholder and the app will auto-generate a strong random secret on first run.
jwt_expiry	How long a login lasts (12h , 7d , etc.).

flexradio — FlexRadio 8600 (SmartSDR)

```

"flexradio": {
  "enabled": true,
  "host_mode": "automatic",
  "host": "",
  "discovery_port": 4992,
  "staleness_timeout_ms": 15000,
  "watchdog_interval_sec": 5,
  "subscriptions": [ ... ],
  "amp_integration": { "amp_on_rf_power": 30, "amp_off_rf_power": 90, "max_rf_power_with_amp": 50 },
  "meters": { ... }
}

```

Key	Meaning
<code>enabled</code>	Turn the FlexRadio integration on/off.
<code>host_mode</code>	<code>automatic</code> = discover the radio on UDP 4992; <code>manual</code> = use <code>host</code> .
<code>host</code>	Radio IP address (only used when <code>host_mode</code> is <code>manual</code>).
<code>discovery_port</code>	FlexRadio discovery port (default 4992).
<code>staleness_timeout_ms</code>	If no data for this long, the link is marked down .
<code>watchdog_interval_sec</code>	How often the staleness check runs.
<code>subscriptions</code>	SmartSDR subscription commands sent on connect. <code>{Client_ID}</code> is substituted.
<code>amp_integration.amp_on_rf_power</code>	RF power (W setting) applied when the amp goes to operate .
<code>amp_integration.amp_off_rf_power</code>	RF power applied when the amp returns to standby .
<code>amp_integration.max_rf_power_with_amp</code>	Hard ceiling on RF power while the amp is engaged.
<code>meters</code>	Map of meter definitions (topic match + display range). Used to scale the gauges. Each meter has <code>min / max</code> and optionally <code>conversion (dbm_to_watts)</code> or <code>global_var</code> .

serial — Palstar LA-1K amplifier

```

"serial": {
  "palstar_lalk_amp": {
    "enabled": true,
    "serial_port": "COM7",
    "baud_rate": 9600,
    "data_bits": 8,
    "parity": "none",
    "stop_bits": 1,
    "response_timeout_ms": 100,
    "protocol": { ... }
  }
}

```

Key	Meaning
<code>enabled</code>	Enable serial control of the amp.
<code>serial_port</code>	Windows COM port (e.g. COM7). Check Device Manager → Ports.
<code>baud_rate</code> , <code>data_bits</code> , <code>parity</code> , <code>stop_bits</code>	Serial line settings (9600/8/N/1).
<code>response_timeout_ms</code>	Per-poll response timeout.
<code>protocol.poll_command</code>	Status poll string (AR1;).
<code>protocol.poll_interval_sec</code>	How often to poll (seconds).
<code>protocol.operate_command</code> / <code>standby_command</code>	Commands for operate (AM1;) / standby (AM2;).
<code>protocol.antenna_select_command</code>	Antenna command; {n} is replaced by 1/2/3.
<code>protocol.frequency_command</code>	Frequency command; {freq_padded_8} is an 8-digit zero-padded kHz value.
<code>protocol.band_codes</code>	Maps the amp's band code field to a label.
<code>protocol.key_status_codes</code>	Maps the amp's key-status field to a label.

If `serialport` cannot load or the port is missing, the amp runs in **simulation mode** and the rest of the app keeps working.

udp — PST Rotator & Palstar HF-Auto Tuner

PST Rotator

```
"pst_rotator": {
  "enabled": true,
  "listen_port": 12001,
  "send_address": "127.0.0.1",
  "send_port": 12000,
  "poll_command": "<PST>AZ?</PST>",
  "poll_interval_sec": 10,
  "set_azimuth_command": "<PST><AZIMUTH>{degrees}</AZIMUTH></PST>",
  "stop_command": "<PST><STOP>1</STOP></PST>",
  "presets": [ { "label": "Caribbean", "value": 100 }, ... ]
}
```

Key	Meaning
<code>listen_port</code>	UDP port the app listens on for azimuth data from PstRotator (12001).
<code>send_address</code> / <code>send_port</code>	Where commands are sent (PstRotator on 127.0.0.1:12000).
<code>poll_command</code> / <code>poll_interval_sec</code>	Azimuth poll string and interval.
<code>set_azimuth_command</code>	Command to turn the rotator; {degrees} is replaced.
<code>stop_command</code>	Emergency stop command.
<code>presets</code>	Heading shortcuts shown in the dashboard dropdown.

Palstar HF-Auto Tuner

```
"palstar_hf_auto_tuner": {
  "enabled": true,
  "listen_port": 13080,
  "send_address": "127.0.0.1",
  "send_port": 12020,
  "broadcast": true,
  "watchdog_timeout_sec": 20,
  "commands": { "select_antenna_1": "<?xml ...>", "mode_auto": "<?xml ...>", [...] },
  "antenna_rules": { "1": { "name": "HexBeam", "force_mode": "bypass"}, [...] }
}
```

Key	Meaning
<code>listen_port</code>	UDP port for incoming XML status (13080).
<code>send_address</code> / <code>send_port</code>	Where XML commands are sent (12020).
<code>broadcast</code>	Send as a UDP broadcast (matches Node-RED's broadcast mode).
<code>watchdog_timeout_sec</code>	If no data within this time, the tuner is marked offline.
<code>commands</code>	Raw XML command strings for antenna/mode selection.
<code>antenna_rules</code>	Friendly name per antenna and the tuner mode automatically forced when it is selected (resonant → bypass, non-resonant → auto).

mqtt — Home Assistant Mosquitto bridge

```
"mqtt": {
  "enabled": true,
  "broker": "192.168.1.54",
  "port": 1883,
  "username": "",
  "password": "",
  "client_id": "hamcontrol_web",
  "birth_topic": "hamcontrol/status",
  "will_topic": "hamcontrol/status",
  "publish_interval_sec": 60,
  "topic_prefix": "hamcontrol/global"
}
```

Key	Meaning
enabled	Turn the MQTT bridge on/off.
broker / port	Mosquitto broker address (your HA at 192.168.1.54:1883).
username / password	MQTT credentials (leave blank if anonymous).
client_id	MQTT client identifier.
birth_topic / will_topic	Online/offline status topics (retained).
publish_interval_sec	How often global state is published.
topic_prefix	Topics published as <prefix>/<key> (power-Supply, radioPower, ampPower, ampStatus, amp, tuner, TXStatus).

home_assistant — Smart plugs, relays & power control

```
"home_assistant": {
  "enabled": true,
  "base_url": "http://192.168.1.54:8123",
  "token": "",
  "timeout_ms": 5000,
  "polling_interval_sec": 15,
  "confirm_delay_ms": 600,
  "entities": {
    "power_supply": { "entity_id": "switch.smart_plug", "state_topic": "" },
    "amplifier": { "entity_id": "switch.smart_plug_2", "state_topic": "" },
    "radio_relay": { "entity_id": "switch.shelly1g4_a085e3c0f2c0", "state_topic": "" }
  }
}
```

Key	Meaning
<code>enabled</code>	Turn Home Assistant control on/off.
<code>base_url</code>	HA URL, e.g. <code>http://192.168.1.54:8123</code> .
<code>token</code>	Home Assistant long-lived access token (Profile → Long-Lived Access Tokens). Required for power control.
<code>timeout_ms</code>	HTTP request timeout for HA service calls and state reads.
<code>polling_interval_sec</code>	How often the server re-reads device states from HA to stay in sync (set <code>0</code> to disable).
<code>confirm_delay_ms</code>	Wait after a switch command before reading the state back to confirm it.
<code>entities.*.entity_id</code>	The HA entity switched by each power button.
<code>entities.*.state_topic</code>	(optional) An MQTT topic that publishes this device's on/off state for real-time updates. Leave blank to rely on REST polling.

How power control works: When you toggle a power button, the server calls the Home Assistant REST API — `POST {base_url}/api/services/switch/turn_on` (or `turn_off`) with `{ "entity_id": "<configured id>" }` and a `Bearer <token>` header. This is exactly what the original Node-RED flow did. The MQTT topics under `topic_prefix` are only status mirrors for HA dashboards; they do **not** switch the devices. All HA calls are logged with `[ha]` / `[power]` prefixes.

State synchronization (UI always reflects reality):

1. **On startup** the server reads the current state of all three devices from HA (`GET /api/states/<entity_id>`) and broadcasts them to every browser, so the UI shows the actual on/off state — even if devices were already on.
2. **After every toggle** the server re-reads the entity and broadcasts the confirmed state (not just the requested value). The toggle button shows a brief **Syncing...** state until HA confirms.
3. **Continuously** the server polls HA every `polling_interval_sec` and, if any `state_topic` is set, subscribes to those MQTT topics for instant updates. Any change made outside the app (HA app, physical switch) is detected and pushed to all connected clients over WebSocket.
4. The Power card shows an **HA SYNCED / SYNCING... / HA ERROR / HA OFF** badge and the time of the last successful sync. Discrepancies between the UI and the real device state are logged as `[ha] STATE CHANGE ...` .

Set `base_url` and the `token` (and confirm the `entity_id`s) on the **Settings** screen. The token is write-only in the UI — it is never sent back to the browser; leave the field blank to keep the saved token.

automation — Behavior toggles

Key	Meaning
<code>amp_rf_power_limiting.enabled</code>	Apply the FlexRadio RF power rules when the amp mode changes.
<code>tuner_antenna_rules.enabled</code>	Auto-force tuner mode when an antenna is selected.
<code>desk_light_tx_rx.enabled</code>	TX/RX desk-light color automation (requires HA).

themes.json

```
{ "active": "dark-amber", "themes": { "dark-amber": { "name": "...", "colors": {
"primary": "#ffb000", ... } } } }
```

- `active` — id of the currently applied theme.
- `themes` — map of theme id → { `name`, `colors` }. Each `colors` object maps a CSS variable name (without the leading `--`) to a hex color.
- Best managed from the **Appearance** tab in the UI, which writes this file.

users.json

```
{ "users": [ { "username": "wb0z", "displayName": "WB0Z", "role": "admin", "password-
Hash": "$2a$...", "disabled": false } ] }
```

- Passwords are stored only as **bcrypt hashes** — never plain text.
- `role` is one of `admin`, `normal`, `viewonly`.
- Manage accounts from the **Users** tab (admin only). To fully reset, delete this file and restart — the default `wb0z` admin is recreated.